

TRAD2026

**TREINAMENTO EM REPRODUTIBILIDADE
PARA ANÁLISE DE DADOS**

Módulo I: Git

Atividade prática



Sumário

1	Comandos Git na prática	3
1.1	Configuração inicial	3
1.2	Criando um repositório local	3
1.3	Criando e adicionando arquivos	4
1.4	Modificando arquivos e criando novos commits	5

1 Comandos Git na prática

Nesta atividade, você irá explorar os principais comandos do Git, aplicando-os em um repositório local. Siga as instruções abaixo para completar a atividade.

1.1 Configuração inicial

Antes de começar a usar o Git, é importante configurar seu nome e endereço de e-mail. Essas informações serão associadas aos commits e ajudarão a identificar suas contribuições no repositório.

```
$ git config --global user.name "Seu Nome"
$ git config --global user.email "seu.email@example.com"
```

Dica.

Se você for colaborar em um projeto com outros desenvolvedores, é recomendável usar o mesmo nome e email que você utiliza no GitHub ou em outras plataformas de hospedagem de código.

Por padrão, o Git cria o branch principal com o nome `master`. No entanto, é uma boa prática utilizar o nome `main` para o branch principal. Para configurar isso, use o comando abaixo:

```
$ git config --global init.defaultBranch main
```

Para verificar se as configurações foram aplicadas corretamente, utilize o comando:

```
$ git config --list
user.name=Seu Nome
user.email=seu.email@example.com
init.defaultBranch=main
```

1.2 Criando um repositório local

O próximo passo é criar um diretório para o seu projeto, use o comando abaixo para criar um novo diretório chamado `projeto` e navegar até ele:

```
$ mkdir projeto  
$ cd projeto
```

Dentro do diretório **projeto**, inicialize um repositório Git com o comando:

```
$ git init  
Initialized empty Git repository in /caminho/para/projeto/.git/
```

Nota.

O comando `git init` inicializa um novo repositório Git, criando o subdiretório `.git`.

1.3 Criando e adicionando arquivos

Agora, crie um arquivo chamado `README.md` e adicione algum conteúdo a ele. Você pode usar o comando `echo` para isso:

```
$ echo "# Meu Projeto" > README.md
```

Para verificar o status do repositório e ver quais arquivos foram adicionados ou modificados, use o comando:

```
$ git status  
On branch main  
No commits yet  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    README.md  
nothing added to commit but untracked files present (use "git add" to track)
```

Para adicionar o arquivo `README.md` ao índice (staging area), use o comando:

```
$ git add README.md
```

Verifique novamente o status do repositório:

```
$ git status  
On branch main
```

```
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md
```

Agora, você pode criar um commit para salvar as alterações no repositório. Use o comando:

```
$ git commit -m "Adiciona README.md"
[main (root-commit) f89925b] Adiciona README.md
1 file changed, 1 insertion(+)
create mode 100644 README.md
```

Dica.

O identificador do commit (hash) é uma sequência única de caracteres que identifica um commit no repositório. No exemplo acima, o hash é f89925b. O seu será diferente, pois é gerado com base no conteúdo do commit e no momento em que ele foi criado.

Veja o histórico de commits com o comando:

```
$ git log
commit f89925becf2e6a17267dadb91e10c9e2391581eb (HEAD -> main)
Author: jvsguerra <jvsguerra@gmail.com>
Date: Sat Aug 22 13:59:27 2026 -0300
    Adiciona README.md
```

1.4 Modificando arquivos e criando novos commits

Agora, vamos modificar o arquivo README.md e criar um novo commit. Adicione mais conteúdo, por exemplo:

```
echo "## Descrição do Projeto" >> README.md
echo "Este é um projeto de exemplo para praticar comandos Git." >> README.md
```

Verifique o status do repositório novamente:

```
$ git status
```



On branch main

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: README.md

Nota.

Note que você modificou o arquivo `README.md`, mas ainda não adicionou essas alterações à staging area. Caso você queira descartar essas alterações e voltar à versão anterior do arquivo, use o comando `git restore README.md`.

